PP-DocLayout: A Unified Document Layout Detection Model to Accelerate Large-Scale Data Construction

Ting Sun, Cheng Cui, Yuning Du, Yi Liu
PaddlePaddle Team, Baidu Inc.
{sunting13, cuicheng01} @baidu.com

Abstract

Document layout analysis is a critical preprocessing step in document intelligence, enabling the detection and localization of structural elements such as titles, text blocks, tables, and formulas. Despite its importance, existing layout detection models face significant challenges in generalizing across diverse document types, handling complex layouts, and achieving real-time performance for large-scale data processing. To address these limitations, we present PP-DocLayout, which achieves high precision and efficiency in recognizing 23 types of layout regions across diverse document formats. To meet different needs, we offer three models of varying scales. ***PP-DocLayout-L*** *is a high-precision model based on the RT-DETR-L detector, achieving 90.4% mAP@0.5 and an end-to-end inference time of 13.4 ms per page on a T4 GPU.* ***PP-DocLayout-M*** *is a balanced model, offering 75.2% mAP@0.5 with an inference time of 12.7 ms per page on a T4 GPU.* ***PP-DocLayout-S*** *is a high-efficiency model designed for resource-constrained environments and real-time applications, with an inference time of 8.1 ms per page on a T4 GPU and 14.5 ms on a CPU. This work not only advances the state of the art in document layout analysis but also provides a robust solution for constructing high-quality training data, enabling advancements in document intelligence and multimodal AI systems. Code and models are available at https://github.com/PaddlePaddle/PaddleX.*

1. Introduction

The rapid development of large language models (LLMs) and multi-modal document understanding systems [10, 11] has led to a significant increase in the demand for high-quality structured training data. Document layout detection, which identifies and localizes structural elements (e.g., text blocks, tables, and figures), plays a pivotal role in transforming raw document images into machine-readable formats. As illustrated in **Figure 1**, layout detection is the foundational step for a variety of downstream tasks, including table recognition, formula recognition, OCR and information extract. For instance, in the case of table recognition, layout detection models accurately locate and define the boundaries of tables within document images, enabling the extraction of table regions for further processing, such as parsing the tabular structure and extracting the underlying data. This structured table data is extremely valuable for applications ranging from data analysis to information retrieval. Similarly, for formula recognition, layout detection models detect and localize formula regions within documents. This allows for the extraction of these regions, which can then be fed into specialized formula recognition systems. The resulting structured formula data not only enhances the machine's understanding of mathematical content but also enriches training datasets, improving models' ability to recognize and interpret formulas in various contexts.

However, despite its potential, existing layout detection models face three critical limitations. (1) Poor generalization across document types. Current approaches predominantly focus on academic papers, resulting in poor performance on other document types such as magazines, newspapers, and financial reports. (2) Inadequate handling of complex layouts. The lack of comprehensive category definitions—such as the absence of separate labels for inline and interline mathematical formulas—necessitates auxiliary models, leading to increased complexity and reduced efficiency. (3) Insufficient processing speed for real-time applications. These challenges impede the effective use of layout detection in practical scenarios, particularly in domains where training large models requires efficient acquisition of abundant massive high-quality data.

To address these challenges, we introduce PP-DocLayout, a unified layout detection model that achieves state-of-the-art accuracy and real-time inference capabilities. PP-DocLayout achieves high-precision recognition and localization of layout regions across diverse document formats including Chinese or English academic papers, research reports, exam papers, books, newspapers, and mag-

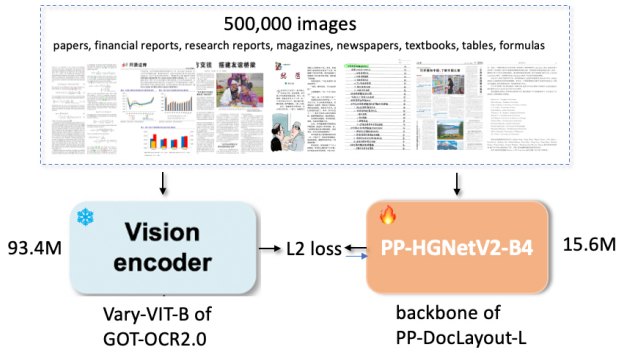

Figure 2. Knowledge Distillation Framework for the backbone of PP-DocLayout-L

3. Method

We introduce PP-DocLayout, a unified detection model achieving state-of-the-art performance through innovations in both data curation and algorithm design. Our approach combines three key improvement strategies.

3.1. Knowledge Distillation Framework

PP-DocLayout-L employs a knowledge distillation [2] paradigm to enhance the performance of document layout understanding as shown in **Figure 2**. In this framework, the visual encoder Vary-VIT-B model of GOT-OCR2.0 [9] acts as the teacher model, which is a well-trained and robust model possessing advanced document understanding capabilities. The student model, in this case, is the PP-HGNetV2-B4 backbone of PP-DocLayout-L, which is designed to learn from the teacher model.

The distillation process involves transferring knowledge from the teacher to the student by aligning their feature representations. Specifically, The teacher network parameters remain frozen while training PP-HGNetV2-B4, leveraging feature-level supervision through a fully-connected layer. Let $\mathbf{F}_{\text{Vary-VIT-B}} \in \mathbb{R}^{B\times D}$ and $\mathbf{F}_{\text{PP-HGNetV2-B4}} \in \mathbb{R}^{B\times P}$ denote the feature tensors from teacher and student networks respectively, where D and P represent their corresponding feature dimensions, B denotes the Batch size. To bridge the dimensional discrepancy ($D \neq P$), we introduce a learnable linear projection $\varphi : \mathbb{R}^P \to \mathbb{R}^D$. The distillation loss is formulated as:

$$\mathcal{L}_{\text{Distill}} = \frac{1}{B}\sum_{i=1}^{B}\left\|\mathbf{F}^{(i)}_{\text{Vary-VIT-B}} - \varphi(\mathbf{F}^{(i)}_{\text{PP-HGNetV2-B4}})\right\|_2^2 \quad (1)$$

The distillation framework was trained on a diverse corpus containing 500000 document samples spanning five domains:

- Mathematical formulas (including equation derivations and symbolic notations)
- Financial documents (reports and balance sheets)
- Scientific literature (arxiv papers in STEM fields)
- Academic dissertations (with complex layout structures)
- Tabular data (statistical reports and spreadsheets)

Training was conducted at 768×768 resolution for 50 epochs using AdamW optimizer ($\beta_1 = 0.9$, $\beta_2 = 0.999$). The distilled PP-HGNetV2-B4 achieves effective feature extraction capabilities with only 15.6 M parameters.

3.2. Semi-supervised Learning

In this section, we present our semi-supervised learning approach employed to enhance the performance of PP-DocLayout-M and PP-DocLayout-S models. This method leverages the high-precision capabilities of the PP-DocLayout-L model to generate pseudo-labels, which are subsequently used to augment the training data for the less complex models.

Pseudo-Label Generation Given an unlabeled document image x_u, we first generate raw predictions using the teacher model PP-DocLayout-L with trained parameters θ_T as follow:

$$P(y|x_u) = f_{\text{T}}(x_u;\theta_T) \quad (2)$$

where $P(y|x_u) \in \mathbb{R}^{N\times C}$ contains prediction scores for N potential regions across $C = 23$ layout categories.

Adaptive Threshold Selection Traditional fixed-threshold approaches often suffer from class imbalance and difficulty levels of learning in different categories. Thus, we propose an adaptive threshold selection method to obtain high-quality pseudo-labels. Our threshold selection strategy explicitly maximizes the F-score on labeled data x_l through a systematic optimization process. For each layout category $c \in \{1, ..., 23\}$, we determine the optimal threshold τ_c^* by solving:

$$\tau_c^* = \arg\max_{\tau\in[0,1]} F_c(\tau; P(y|x_l)) \quad (3)$$

where $F_c(\tau)$ denotes the F1-score computed on the validation set $\mathcal{V}$ when using threshold τ for class c.

After determining the optimal thresholds for each class, we generate pseudo-labels for the unlabeled data. For each potential region in the document image x_u, a pseudo-label is assigned if the prediction score exceeds the corresponding optimal threshold τ_c^*.

Pseudo-Label Generation and Training Using the optimized thresholds $\{\tau_1^*, \tau_2^*, \ldots, \tau_{23}^*\}$, the pseudo-labels $\hat{y}_u$ for the unlabeled document image x_u are defined as:

$$\hat{y}_{u,i} = \begin{cases} 1, & \text{if } P(y_{i,c}|x_u) > \tau_c^* \text{ for any } c \\ 0, & \text{otherwise} \end{cases} \quad (4)$$

where $\hat{y}_{u,i}$ indicates whether the i-th region is assigned a pseudo-label for any category. The pseudo-labeled data,

Our Method	DocLayout-YOLO
Document Title	Title
Paragraph Title	Title
Text	Plain Text
Page Number	Abandon
Abstract	Plain Text
Content	Plain Text
Reference	Plain Text
Footnote	Abandon
Header	Abandon
Footer	Abandon
Algorithm	Plain Text
Formula	Isolate Formula
Formula Number	Formula Number
Image	Figure
Image Caption	Figure Caption
Table	Table
Table Caption	Table Caption
Chart	Figure
Chart Caption	Figure Caption
Seal	Figure
Header Image	Abandon
Footer Image	Abandon
Aside Text	Abandon

Table 1. Comparison of categories recognized by our method and another advanced algorithm DocLayout-YOLO [14].

along with the real labeled data, form a comprehensive training set that enhances the learning process of the student models PP-DocLayout-S and PP-DocLayout-M. By integrating the high-quality pseudo-labels with the labeled data, the models can generalize better, learning robust features that improve document layout detection.

4. Experimental Results

4.1. Dataset

we collected a comprehensive dataset of document images encompassing a wide variety of types such as Chinese and English academic papers, magazines, newspapers, research reports, exam papers, handwritten notes, contracts, and books. This diverse dataset ensures the robustness and generalizability of our model across different document formats and structures. The dataset comprises 30,000 images for training and 500 images for evaluation. Images are collected from Baidu image search and public datasets, including Doclaynet [6] and PublayNet [15]. Images are annotated with 23 common layout categories and the distribution of these categories is detailed in the Table 5 in the appendix.

As shown in **Table 1**, our method defines a more comprehensive and fine-grained set of categories compared to DocLayout-YOLO [14]. While DocLayout-YOLO simplifies many document elements into broad classes such as "title," "text," and "figure," our method distinguishes between semantically meaningful elements like document titles, paragraph titles, page numbers, headers, footers, and footnotes. This granularity enables better parsing of the document's hierarchical structure and logical relationships. Additionally, our method accurately identifies and categorizes high-value elements such as formulas, charts, and seals, which DocLayout-YOLO either misclassifies or ignores (e.g., labeling them as "abandon" or "figure"). This comprehensive categorization supports a wider range of downstream tasks, including document understanding, information extraction, and format conversion.

4.2. Implementation Details

The **PP-DocLayout-L** model is built upon the RT-DETR-L [13] object detection architecture and utilizes a pre-trained PPHGNetV2-B4 model that has undergone knowledge distillation. The training was configured with a constant learning rate of 0.0001. The model was trained for 100 epochs with a batch size of 2 per GPU, using a total of 8 GPUs, resulting in a total training time of approximately 26 hours on NVIDIA V100 GPUs. The **PP-DocLayout-M** and **PP-DocLayout-S** models are based on the PicoDet-M and PicoDet-S [12] object detection architecture, respectively. Both models were trained for 100 epochs with a batch size of 2 per GPU, utilizing a total of 8 GPUs. The learning rates were set to for 0.02 PP-DocLayout-M and 0.06 for PP-DocLayout-S, and were dynamically adjusted using the CosineDecay [5] learning rate scheduler.

4.3. Main Results

Table 2 presents the performance and specifications of different variants of the PP-DocLayout model. The table highlights the trade-offs between accuracy, inference speed, and model size for each model variant.

The PP-DocLayout-L model achieves the highest accuracy with a mean Average Precision (mAP) of 90.4% at an IoU threshold of 0.5. However, this accuracy comes with a model size of 30.94 million parameters and inference times, taking 13.39 milliseconds on a T4 GPU, about 74.6 FPS and approximately 759.76 milliseconds on a CPU, which translates to about 1.32 FPS. Referring to **Figure 4** in the appendix, we provide additional visualization results to further demonstrate the effectiveness of our model across a diverse range of document types and layouts. Specifically, we visualize the performance of our method on documents such as *papers*, *magazines*, *newspapers*, *research reports*, *books*, *notebooks*, *contracts* and *test papers*. The visualizations clearly show that our model accurately identifies and categorizes diverse elements.

The PP-DocLayout-S model offers a significantly smaller model size of 1.21 million parameters with faster inference times 8.11 milliseconds on a T4 GPU, about 123 FPS and 14.49 milliseconds on a CPU, which translates to

Model Name	mAP@0.5 (%)	GPU Inference (ms)	CPU Inference (ms)	Params (M)
PP-DocLayout-L	90.4	13.39	759.76	30.94
PP-DocLayout-M	75.2	12.73	59.82	5.65
PP-DocLayout-S	70.9	8.11	14.49	1.21

Table 2. Model performance and specifications. *Note:* GPU inference times are based on an NVIDIA Tesla T4 machine. CPU inference speeds are based on an Intel(R) Xeon(R) Gold 6271C CPU @ 2.60GHz, with 8 threads and FP16 precision.

approximately 69.04 FPS. Despite its compactness, it maintains a respectable mAP of 70.9%.

The PP-DocLayout-M model strikes a balance between the two extremes, achieving an mAP of 75.2%. It has a moderate model size of 5.65 million parameters, with inference times of 12.73 milliseconds on a T4 GPU and about 59.82 milliseconds on a CPU, translating to approximately 16.72 FPS.

These results illustrate the trade-offs inherent in model design, where increases in accuracy often come at the expense of model size and inference speed. The choice of model may thus depend on the specific requirements of accuracy, computational resources, and latency in practical applications.

4.4. Qualitative Analysis

In this section, we compare the results of our proposed method with the advanced method DocLayout-YOLO [14] through visualization. Due to differences in label categories, traditional quantitative metrics cannot be directly applied. Therefore, we employ visualization techniques to present the performance of each method enabling an intuitive comparison.

The visualization results are shown in **Figure 3**. The first row presents the results of DocLayout-YOLO [14], while the second row shows the results of our method. From the first column of images, it can be observed that our results include elements such as document titles, abstracts, paragraph titles and text, which are essential for understanding the semantic hierarchy and logical structure of the document. In contrast, DocLayout-YOLO categorizes these elements into only two broad classes, "title" and "plain text," which limits its ability to parse the document's semantic layers effectively. Additionally, our PP-DocLayout can accurately locate page numbers, headers, footers, and footnotes, whereas DocLayout-YOLO often classifies these as "abandon," disregarding their potential value.

The second column highlights the difference in formula recognition. Our method is capable of identifying both inline and block-level formulas, which is crucial for downstream tasks such as PDF-to-Markdown conversion. In contrast, DocLayout-YOLO struggles to recognize inline formulas, focusing only on prominent block-level formulas, which hinders its utility in tasks requiring full-text recognition.

In the third column, the comparison demonstrates our method's superior performance in handling handwritten notes. While our approach correctly identifies and categorizes handwritten content, DocLayout-YOLO misclassifies it as "figure" failing to capture its textual significance.

Finally, the fourth column illustrates our method's ability to distinguish between natural images, charts, and seals. Charts and seals are particularly important as they often contain high-value information, and our method ensures they are categorized separately. DocLayout-YOLO, on the other hand, does not make this distinction, potentially overlooking critical details.

Overall, our method provides a more granular and accurate representation of document elements, enabling better semantic understanding and supporting a wider range of downstream tasks compared to DocLayout-YOLO.

4.5. Ablation Study

In order to evaluate the impact of semi-supervised learning and knowledge distillation on model performance, we conducted a series of ablation experiments using the PP-DocLayout model variants. We compared the performance of each model with and without these techniques, measuring the mean Average Precision (mAP) at an IoU threshold of 0.5.

Knowledge Distillation We examined the effect of knowledge distillation on the PP-DocLayout-L variant as shown in **Table 3**, The use of knowledge distillation yielded an mAP improvement from 89.3% to 90.4%, demonstrating a positive impact on model accuracy.

Algorithm Name	Knowledge Distillation	mAP@0.5 (%)
PP-DocLayout-L	No	89.3
PP-DocLayout-L	Yes	90.4 (+1.1)

Table 3. Results of the ablation study on the effect of knowledge distillation.

Semi-supervised Learning As shown in **Table 4**, for the PP-DocLayout-M and PP-DocLayout-S models, incorporating semi-supervised learning significantly enhanced performance. Specifically, the mAP for PP-DocLayout-M improved from 73.8% to 75.2%, reflecting a 1.4% increase.